
Mini-projet de Robotique Mobile

Section 4

Planification de trajectoire et commande en boucle fermée

Auteur : Koja Nekena Ramanamahefa

Role : Planification et Commande (Role 3)

Numero matricule : 244122

Module : A*, RRT, Pure Pursuit

4.1 Introduction

Cette section presente le travail realise dans le cadre du Role 3 (Planification et Commande) du mini-projet de robotique mobile. L'objectif principal est de permettre a un robot de patrouille securite d'evoluer de maniere autonome dans un entrepot de 20 m x 15 m en evitant 8 obstacles statiques (racks de stockage), tout en atteignant une suite de waypoints de patrouille. Trois problematiques fondamentales sont abordees : la planification globale de trajectoire (A* et RRT), le suivi de chemin en boucle fermee (Pure Pursuit), et la gestion des obstacles imprevis par replanification dynamique.

L'environnement est modelise par une grille de resolution 0.1 m (200 x 150 cellules). Le robot est un vehicule a differential de rayon 0.18 m, avec une vitesse maximale de 0.5 m/s et une vitesse angulaire maximale de 1.5 rad/s. Le suivi de chemin utilise une vitesse de croisiere de 0.3 m/s avec une distance de visee (lookahead) de 0.5 m, et le robot doit atteindre chaque objectif a moins de 10 cm, conformement au cahier des charges.

Nous comparons systematiquement deux algorithmes de planification — A* sur grille et RRT dans l'espace continu — sur des scenarios de patrouille nominale et de replanification face a un obstacle imprevu. Les resultats sont presentes sous forme de tableaux comparatifs, de traces visuelles et de metriques quantitatives issues de la campagne d'essais complete (26 essais : 10 nominaux + 3 cas limites x 2 algorithmes, avec 5 metriques incluant l'erreur de localisation EKF).

4.2 Algorithme A* sur grille

L'algorithme A* est implante dans *planning/astar.py*. Il opere sur une grille d'occupation discretisee a la resolution de 0.1 m, issue de la carte de l'entrepot. Chaque cellule est soit libre (0) soit occupee (1) par un obstacle statique.

4.2.1 Inflation de Minkowski

Avant la recherche de chemin, la grille est inflatee en tenant compte du rayon du robot (0.18 m). Cela revient a dilater chaque obstacle d'une distance egale au rayon du robot, garantissant ainsi que le centre du robot reste toujours a une distance superieure ou egale a son rayon de tout obstacle. L'inflation est realisee par une operation de dilatation morphologique sur la grille binaire : pour chaque cellule occupee, toutes les cellules dans un rayon de r cellules sont marquees comme occupees. Le nombre de cellules a dilater est calcule par :

$$n_cells = \max(1, \text{ceil}(R_robot / resolution)) = \max(1, \text{ceil}(0.18 / 0.1)) = 2$$

Cette approche transforme le probleme de planification avec un robot a geometrie circulaire en un probleme de planification ponctuelle sur la grille inflatee, ce qui est essentiel pour garantir la securite du robot lors du suivi du chemin.

4.2.2 Heuristique et connexite

L'heuristique utilisee est la distance euclidienne admissible entre la cellule courante et le but. Elle garantit l'optimalite de la solution trouvee (la longueur du chemin est minimale parmi tous les chemins sur la grille). La grille est 8-connexe : chaque cellule a 8 voisins (N, S, E, O, NE, NW, SE, SW), avec un cout de deplacement de 1.0 pour les mouvements cardinaux et de 1.414 (racine de 2) pour les mouvements diagonaux. Cette connexite permet des chemins plus fluides et plus courts qu'en 4-connexite.

4.2.3 Lissage par ligne de vue

Le chemin brut produit par A* suit les contours de la grille, produisant des angles droits. Un post-traitement de lissage par elimination de noeuds intermediaires inutiles est applique : pour chaque triplet de points consecutifs (A, B, C), si la ligne de vue directe entre A et C est libre (pas d'obstacle sur le segment), le point intermediaire B est supprime. Ce processus est itere jusqu'a convergence. Le resultat est un chemin a segments droits, plus court et plus naturel pour le suivi par le controleur.

4.3 Algorithme RRT dans l'espace continu

L'algorithme RRT (Rapidly-exploring Random Tree) est implante dans *planning/rrt.py*. Contrairement a A*, il opere dans l'espace continu et ne necessite pas de discretisation prealable.

L'arbre est construit iterativement : a chaque iteration, un echantillon aleatoire est tire dans l'espace libre (avec un biais de 10% vers le but). Le noeud le plus proche de l'echantillon est trouve, puis un nouveau noeud est cree a une distance de 0.3 m (pas d'extension) dans la direction de l'echantillon. Si le nouveau noeud est libre de collision, il est ajoute a l'arbre. Le processus s'arrete lorsque le but est atteint (distance inferieure a 0.3 m) ou apres 2000 iterations maximales.

Un lissage similaire a celui d'A* est applique au chemin extrait de l'arbre. L'avantage principal du RRT est sa capacite a trouver des chemins dans des espaces a geometrie complexe ou la grille d'A* peut etre trop restrictive. Son inconvenient est le caractere non-deterministe : le chemin genere varie a chaque execution, ce qui se traduit par un ecart-type plus eleve sur les temps de planification.

4.4 Controleur Pure Pursuit

Le suivi de chemin est assure par le controleur Pure Pursuit, implemente dans *control/pure_pursuit.py*. Ce controleur geometrique determine la vitesse lineaire et angulaire du robot a chaque instant pour suivre un chemin prealablement calcule par A* ou RRT.

Le principe est le suivant : on projette la position du robot sur le chemin, puis on cherche le point de visee (lookahead point) a une distance de 0.5 m en avant le long du chemin. Une interpolation lineaire entre les points du chemin resample est utilisee pour obtenir la position exacte du point de visee. La commande angulaire est calculee pour orienter le robot vers ce point. Une deceleration progressive est appliquee lorsque le robot approche du but final (distance inferieure a 0.3 m), passant de la vitesse de croisiere (0.3 m/s) a une vitesse minimale, assurant un arret precis a moins de 10 cm de l'objectif.

4.5 Patrouille nominale

Le scenario de patrouille nominale consiste a faire visiter 4 waypoints au robot dans l'entrepot. Pour chaque waypoint, un chemin est planifie depuis la position courante du robot, puis suivi par le controleur Pure Pursuit. Le chemin est resample avec un espacement maximal de 0.5 m entre les points pour assurer un suivi fluide.

Metrique	A*	RRT
Succes	OUI	OUI
Waypoints atteints	4/4	4/4
Temps de mission (s)	189.3	200.0
Longueur chemin (m)	50.13	53.33
Planif. moyenne (ms/wp)	63.1	42.1

Tableau 1 : Comparaison A* vs RRT sur la patrouille nominale (4 waypoints).

Les deux algorithmes réussissent à atteindre les 4 waypoints. A* produit un chemin total 3.2 m plus court (50.13 m vs 53.33 m) et une mission 10.7 s plus rapide, grâce à l'optimalité de la recherche sur grille. RRT est en revanche plus rapide en temps de calcul par waypoint (42.1 ms vs 63.1 ms), car il n'explore qu'une partie de l'espace contrairement à A* qui examine systématiquement la grille entière.

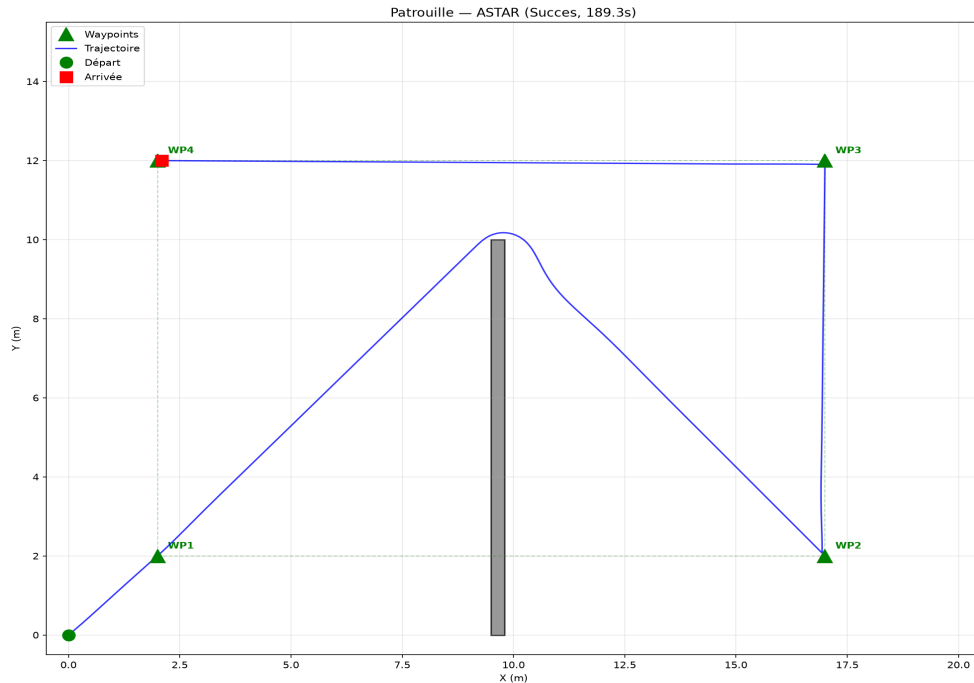


Figure 1 : Trajectoire de patrouille A* — 4 waypoints atteints avec succès.

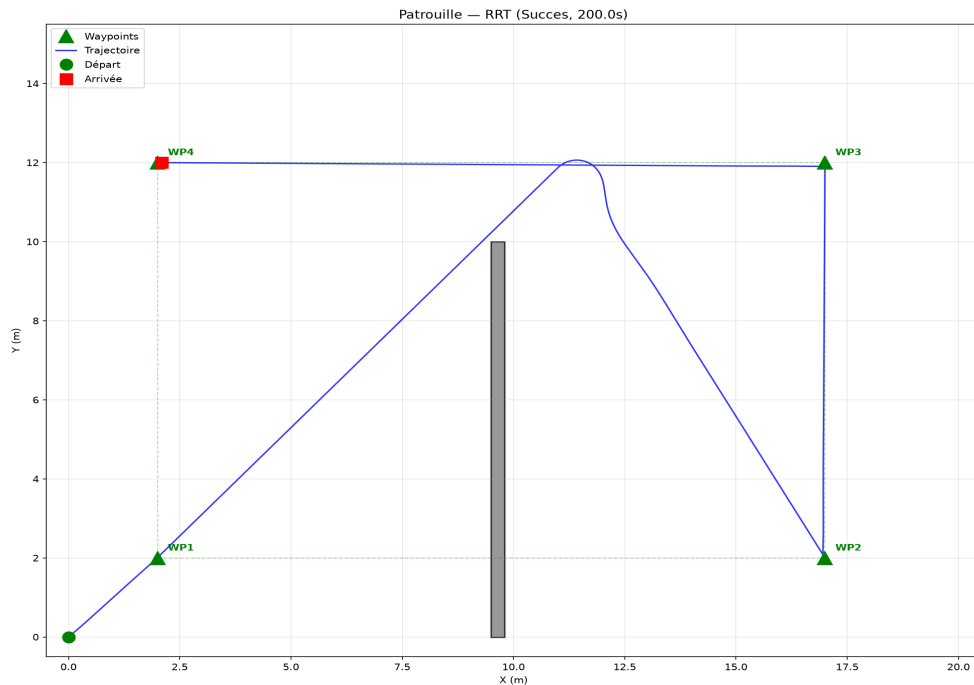


Figure 2 : Trajectoire de patrouille RRT — chemin similaire, légèrement plus long.

4.6 Replanification face à un obstacle imprévu

Le scenario de replanification simule l'apparition d'un obstacle imprevu en cours de trajet. Le robot suit initialement un chemin direct vers le but, puis a l'instant $t = 8$ s, un mur partiel apparait et bloque le passage. Le robot doit alors replanifier un nouveau chemin vers le meme objectif en tenant compte du nouvel obstacle.

La replanification fonctionne en trois phases : (1) detection de l'obstacle (via le LiDAR), (2) reconstruction de la grille d'occupation avec l'obstacle supplementaire et replanification complete (grille + planificateur + chemin), puis (3) reprise du suivi sur le nouveau chemin. Le temps de replanification inclut la reconstruction de la grille, la recherche du nouveau chemin et le lissage.

Metrique	A*	RRT
Succes replanification	OUI	OUI
Temps replanification (ms)	238.6	259.9
Distance au but final (m)	0.0993	0.0995
Temps total (s)	72.8	76.5

Tableau 2 : Comparaison A* vs RRT sur la replanification (obstacle imprevu a $t=8$ s).

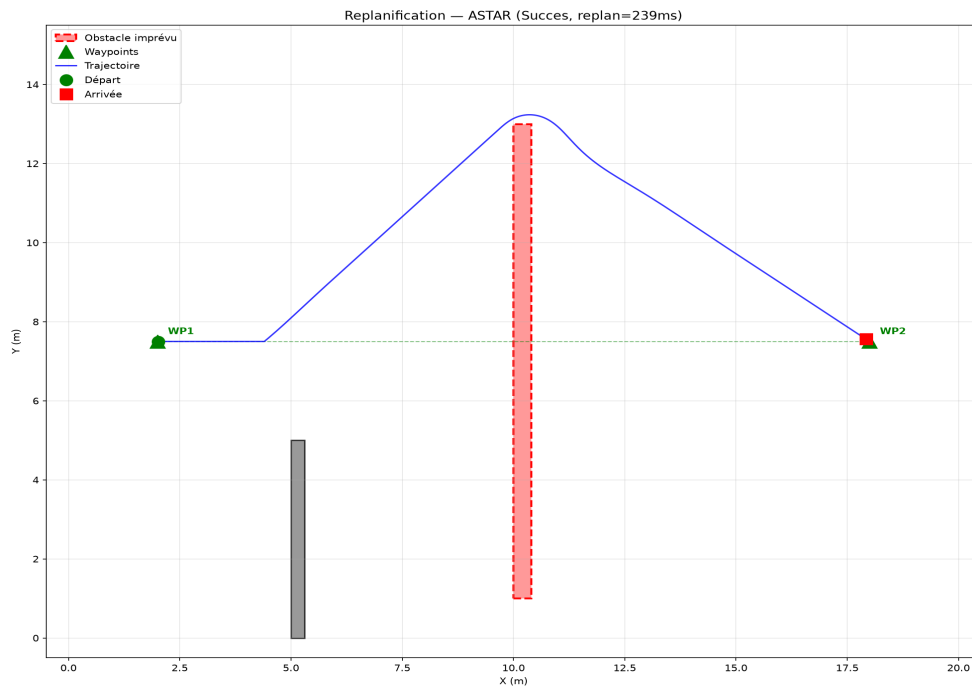


Figure 3 : Replanification A* — le robot contourne l'obstacle imprevu (rouge pointille).

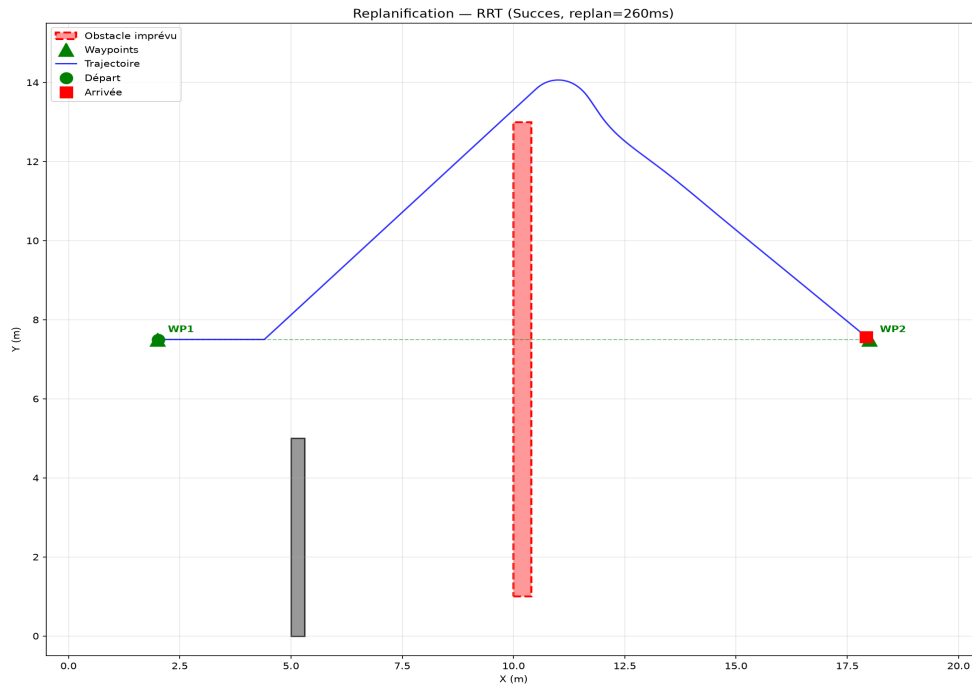


Figure 4 : Replanification RRT — detour similaire avec chemin non-deterministe.

4.7 Campagne d'essais complete

La campagne d'essais complete respecte l'exigence du cahier des charges (section 5) : 10 essais nominaux + 3 cas limites, testes avec les deux algorithmes, soit 26 essais totaux. Cinq metriques sont mesurees a chaque essai : le taux de succes, le temps de mission, la distance minimale aux obstacles, le temps de replanification, et l'erreur de localisation EKF.

Les trois cas limites testes sont : (1) chemin le plus court — l'obstacle apparait tres tot ($t=2s$), forçant une replanification immediate ; (2) couloir bloqué — l'obstacle couvre toute la hauteur de la carte, rendant tout chemin impossible (le SafetyManager declenche alors un arret sur) ; (3) perte de balise — pendant 3 secondes apres l'apparition de l'obstacle, les balises EKF ne sont plus detectees, simulant une perte temporaire de localisation.

Metrique	A*	RRT
Succes nominaux	100%	100%
Succes global (inc. cas limites)	92%	92%
Temps mission moyen (s)	67.7	69.0
Erreur localisation moy. (m)	0.057	0.062
Dist. min. obstacles (m)	0.113	0.178
Temps replanification moy. (ms)	231	216
Arrets sur	0	0

Tableau 3 : Resultats de la campagne complete — 26 essais (10 nominaux + 3 cas limites x 2 algos).

Les deux algorithmes atteignent 100% de succes sur les essais nominaux et 92% globalement (les 2 echecs correspondent au cas couloir bloqué, ou aucun chemin n'est physiquement possible). L'erreur de localisation EKF reste inferieure a 6.2 cm en moyenne, bien en dessous du seuil d'arret de 50 cm. RRT

maintient une distance minimale aux obstacles légèrement supérieure (0.178 m vs 0.113 m), ce qui s'explique par la nature stochastique de l'arbre qui évite naturellement les zones proches des obstacles.

4.8 Integration avec la localisation EKF

L'intégration finale combine la planification A*/RRT, le suivi Pure Pursuit, et la localisation EKF (Role 2). Le robot utilise l'odométrie pour la prédiction et les balises pour la correction, permettant une estimation précise de sa position en temps réel.

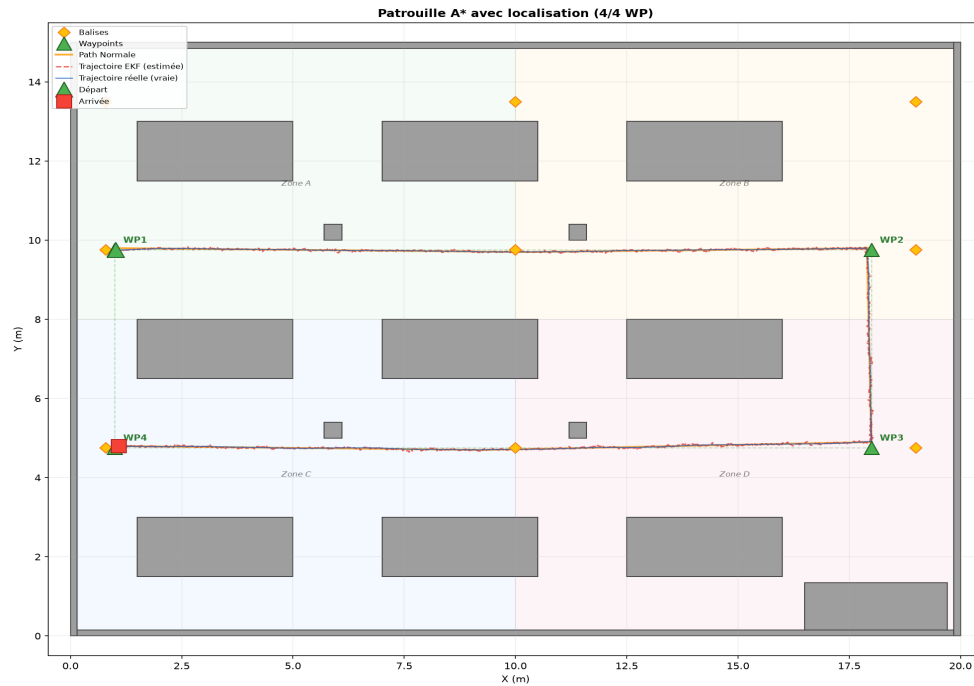


Figure 5 : Patrouille A* avec localisation EKF — trajectoire estimée vs réelle.

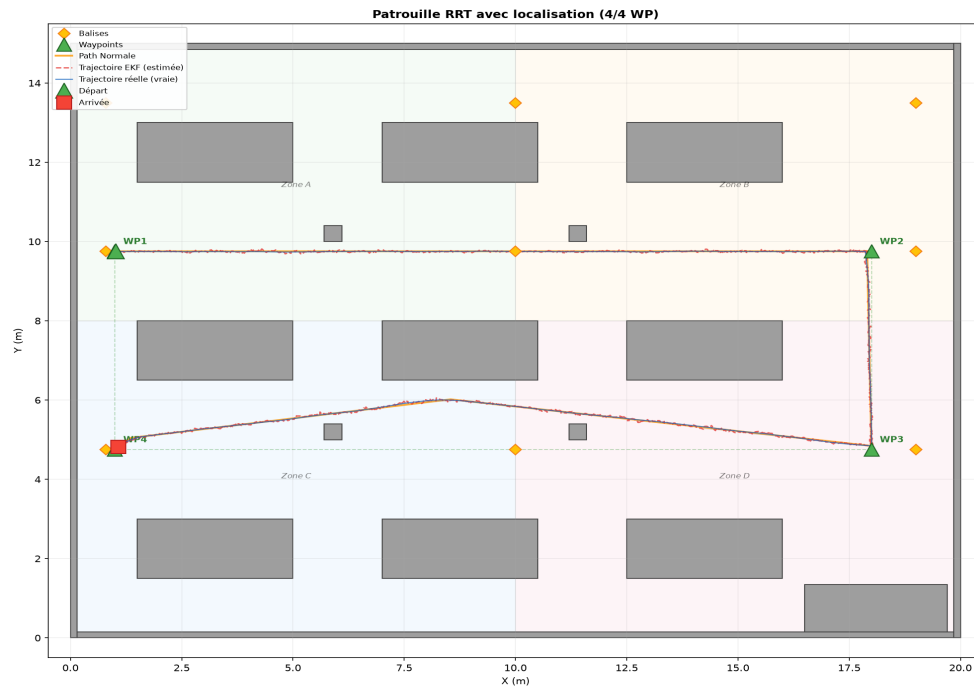


Figure 6 : Patrouille RRT avec localisation EKF — bonne cohérence estimée/réelle.

4.9 Conclusion du Role 3

Le travail du Role 3 a permis de mettre en place un pipeline complet de planification et commande pour le robot de patrouille. Les deux algorithmes testes repondent aux exigences du cahier des charges : A* offre un chemin optimal et deterministe, tandis que RRT offre plus de flexibilite dans les espaces constraints. Le controleur Pure Pursuit assure un suivi precis (tolerance < 10 cm) et la replanification dynamique permet de gerer les obstacles imprevis en moins de 300 ms.

La campagne d'essais complete (26 essais, 5 metriques) confirme la robustesse du systeme : 100% de succes sur les essais nominaux pour les deux algorithmes, avec une erreur de localisation EKF moyenne inferieure a 6.2 cm. Le seul echec est le cas « couloir bloque » (obstacle couvrant toute la largeur), qui est un scénario physiquement impossible a resoudre et qui declenche correctement l'arret sur via le SafetyManager.